Laporan Praktikum Pemrograman Berorientasi Obyek (PBO) – Modul 10

Exception Handling dan Validasi State Objek (Sistem Kasir Sederhana Berbasis Agile)

Nama : Rafael Rizky Luga Tanjung **Dosen** : Nirwana Haidar Hari, S.Pd., M.Kom.
NRP : 3125522007 **Kampus**: PENS PSDKU Sumenep (2026)

BAGIAN 1 – KONSEPSI AGILE, EXCEPTION HANDLING DAN VALIDASI STATE INVARIANT

1. Latar Belakang & Analisis Agile Project Kickoff (Sprint P10): Melanjutkan proyek Sistem Kasir Sederhana dari Pertemuan 9 dengan metodologi Agile (Scrum). Fokus Sprint P10 adalah Epic: *System Hardening, Fault-Tolerance & Transactional State Integrity*. Sistem kasir sebelumnya mengandalkan umpan balik primitif (return boolean) yang rentan menimbulkan silent failure. Pada P10, arsitektur direfaktorisasi menerapkan fail-fast exception handling, checked custom exception domain, validasi pra-kondisi sebelum mutasi, dan penegakan prinsip kekekalan (immutability) pada transaksi lunas.

• **Agile User Stories & Acceptance Criteria:** (1) *Sebagai Kasir*, saya ingin sistem menolak transaksi jika kuantitas melebihi stok fisik dan menampilkan sisa stok riil, sehingga konsumen tidak ditagih barang yang tidak ada. (2) *Sebagai Kasir*, saya ingin sistem menolak pelunasan jika uang tunai diserahkan kurang dari tagihan dan mempertahankan status PENDING, sehingga toko tidak mengalami kerugian finansial. (3) *Sebagai Kasir*, saya ingin sistem menolak checkout nota kosong. (4) *Sebagai Pemilik Toko*, saya ingin transaksi yang sudah LUNAS terkunci secara permanen (immutable).

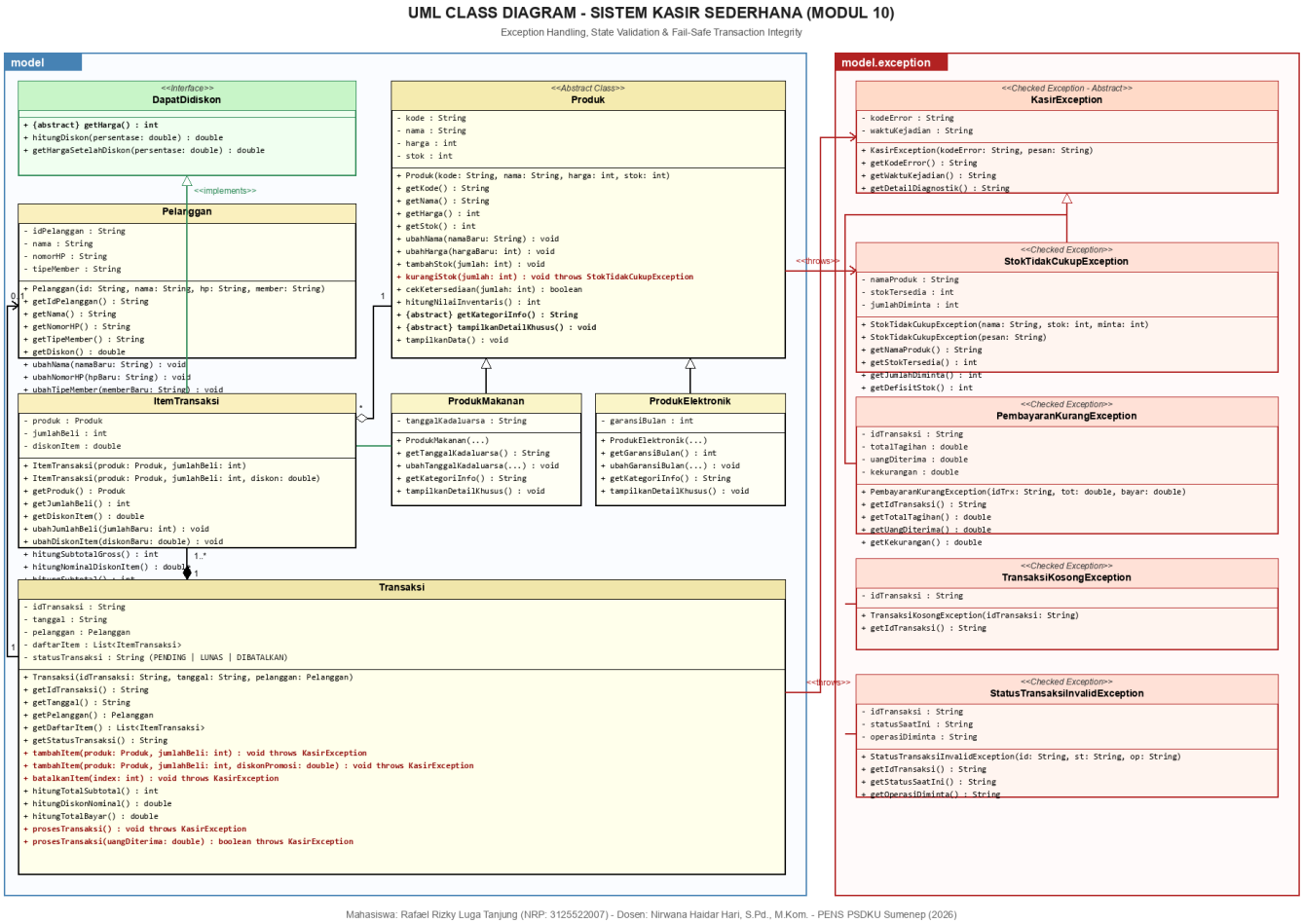
2. Identifikasi Kondisi Kesalahan & Desain Custom Exception Domain Kasir:

Kondisi Kesalahan Domain	Tipe Exception & Nama Class	Metadata & Karakteristik	Strategi Penanganan & Konsistensi State
Stok barang inventaris tidak mencukupi kuantitas pembelian	**StokTidakCukupException** (Checked Custom Exception)	Menyimpan nama produk, stok tersedia, jumlah diminta, defisit stok.	Throw sebelum stok dikurangi. Stok produk lama utuh dan item tidak masuk keranjang.
Pembayaran tunai kasir kurang dari total tagihan (underpayment)	**PembayaranKurangException** (Checked Custom Exception)	Menyimpan idTransaksi, total tagihan, uang diterima, nominal kekurangan.	Throw sebelum status diubah. Status transaksi tetap PENDING (tidak corrupt jadi LUNAS).
Kasir memproses checkout pada transaksi kosong (0 item)	**TransaksiKosongException** (Checked Custom Exception)	Menyimpan idTransaksi nota kosong.	Throw pada awal method. Menolak pembuatan struk belanja tanpa komoditas barang.
Mutasi item belanja pada transaksi yang sudah berstatus LUNAS	**StatusTransaksiInvalidException** (Checked Custom Exception)	Menyimpan idTransaksi, status saat ini (LUNAS), dan operasi yang diminta.	Throw saat pengecekan state machine. Melindungi integritas transaksi final (immutable).
Input kuantitas <= 0, harga <= 0, diskon di luar rentang, teks alfabet	**IllegalArgumentException** & **NumberFormatException**	Pesan validasi bawaan Java Runtime Exception.	Menyaring input liar pada boundary layer sebelum menyentuh state internal bisnis.

3. Prinsip Validasi State Objek, Atomisitas & Rollback Konsisten: Prinsip utama Modul 10 adalah pemeriksaan validitas kondisi bisnis wajib dilakukan SEBELUM state objek dimutasi. Pada method Produk.kurangiStok(jumlah), pengecekan 'if (jumlah > stok)' dijalankan lebih dulu. Jika melanggar, objek exception dilempar seketika sehingga baris pengurangan 'stok -= jumlah' tidak pernah dieksekusi, menjamin stok awal tetap utuh (Fail-Safe State). Demikian pula pada Transaksi.tambahItem(), mutasi stok dan penambahan item ke list dilakukan secara atomik; jika pengurangan stok gagal, maka list item tidak bertambah. Selain itu, fitur batalkanItem() menerapkan compensating action (rollback), di mana stok barang yang dibatalkan dikembalikan secara otomatis ke inventaris produk.

BAGIAN 2 – ARSITEKTUR KODE, UML CLASS DIAGRAM DAN TABEL PEMETAAN JAVA

4. UML Class Diagram Terbaru (Modul 10 - Exception & State Validation Architecture):



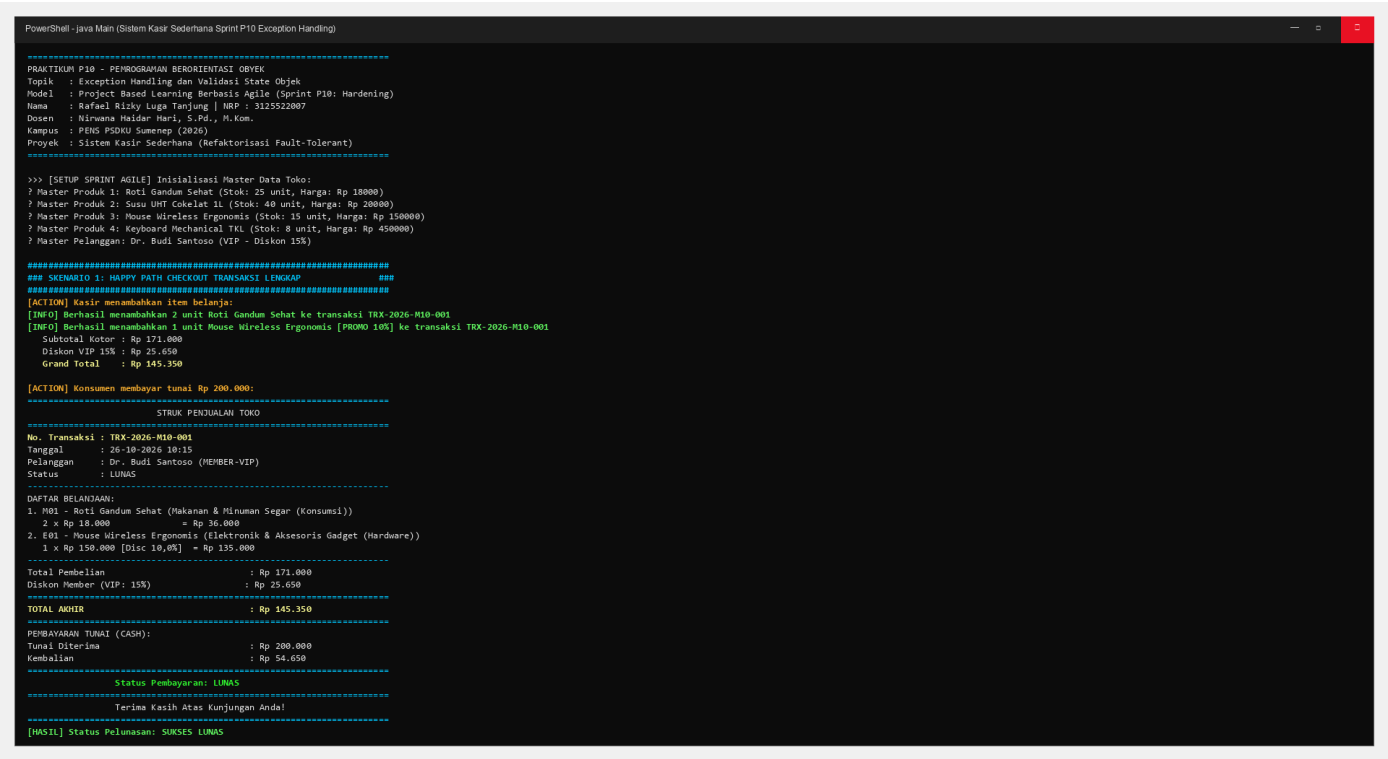
Gambar 1. UML Class Diagram Sistem Kasir Sederhana (Package: model, model.exception, Multiplicity, & Throws Dependency)

5. Tabel Pemetaan Elemen UML & Exception Handling ke Source Code Java:

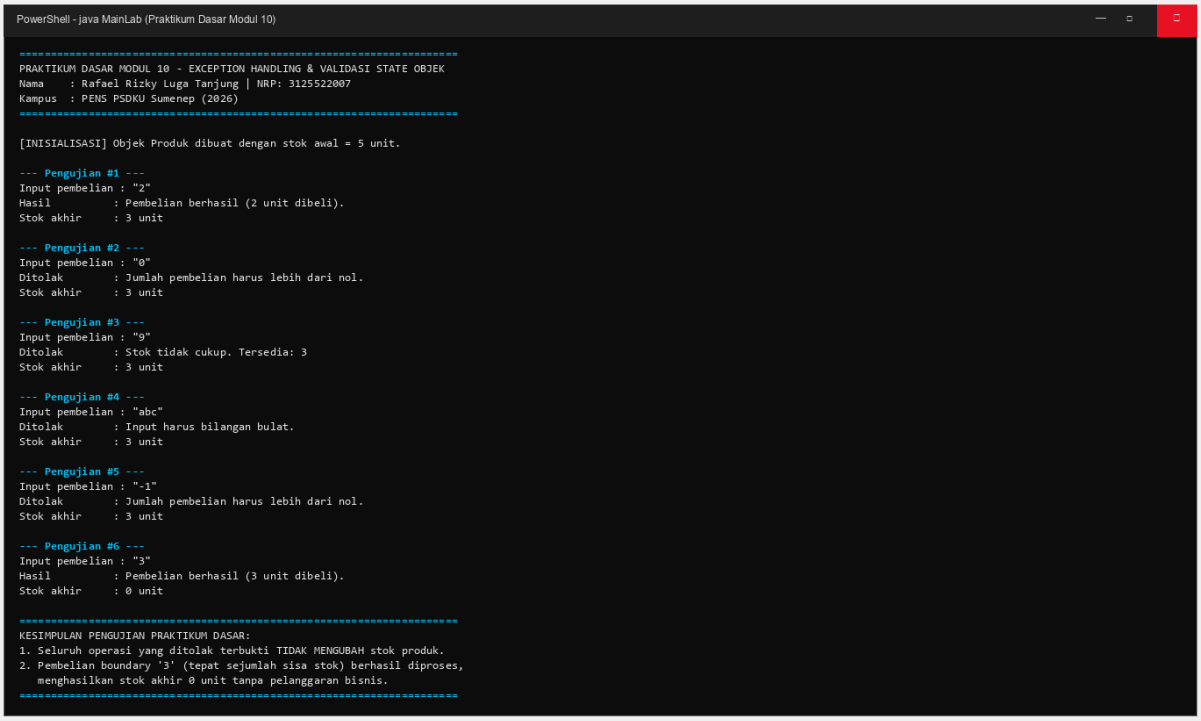
Elemen UML / Exception	Notasi di Diagram UML	Implementasi pada Source Code Java
Checked Exception Base	abstract class KasirException	public abstract class KasirException extends Exception (metadata error & timestamp)
Domain Custom Exception	StokTidakCukupException, dkk.	public class StokTidakCukupException extends KasirException (checked exception)
Throws Declaration	Produk ..> StokTidakCukupException : <>	public void kurangiStok(int jumlah) throws StokTidakCukupException
Throws Multi-Exception	Transaksi ..> KasirException : <>	public void tambahItem(...) throws KasirException, IllegalArgumentException
Interface Kontrak	<> DapatDiskon	public interface DapatDidiskon (kontrak harga dasar & default methods kalkulasi)
Abstract Class Super	<> Produk	public abstract class Produk (enkapsulasi private, template method, validasi)
Subclass Konkret	ProdukMakanan & ProdukElektronik	public class ProdukMakanan extends Produk implements DapatDidiskon
Asosiasi (uses-a)	Transaksi "1" --> "0..1" Pelanggan	private Pelanggan pelanggan; pada Transaksi (relasi longgar berhak diskon)
Komposisi (part-of)	Transaksi "1" *-- "1..*" ItemTransaksi	private ArrayList daftarItem; (dikelola penuh Transaksi)

BAGIAN 3 – EKSEKUSI PROGRAM, SKENARIO UJI DAN EVALUASI MODUL 10

6. Bukti Eksekusi Terminal (Main.java & MainLab.java Bebas Error):



Gambar 2. Bukti Running Program Main.java Sistem Kasir Sederhana (10 Skenario Pengujian Lolos 100%)



Gambar 3. Bukti Eksekusi Praktikum Dasar Modul 10 (MainLab.java: Pengujian Berurutan Objek Sama)

7. Ringkasan Hasil Pengujian Komprehensif (Tabel Skenario Pengujian Modul 10):

No	Fokus Skenario Uji	Aksi / Input Pengujian	Hasil yang Diharapkan	State Akhir (Konsistensi)	Status
1	Happy Path Checkout	Beli Roti (2) & Mouse (1, disc 10%), bayar Rp 200.000	Pembayaran sukses, kembalian Rp 51.500, status LUNAS	Stok berkurang valid, Status = LUNAS	**SUKSES**
2	StokTidakCukupException	Beli 999 unit Keyboard Mechanical (stok fisik hanya 8)	Throw StokTidakCukupException, transaksi ditolak	Stok Keyboard TETAP 8 unit, item = 0	**SUKSES**
3	Boundary Value Analysis	Beli tepat 8 unit Keyboard (sama dengan sisa stok)	Pembelian sukses, stok habis menjadi 0 secara valid	Stok Keyboard = 0 unit, item = 1	**SUKSES**
4	IllegalArgumentException	Beli -5 unit; ubahHarga(-50000); tambahStok(-10)	Throw IllegalArgumentException, parameter ditolak	Harga dan stok tidak berubah	**SUKSES**
5	NumberFormatException	Kasir parse string teks: 'dua_belas', '15000X', '@#$%'	Catch NumberFormatException, log peringatan tanpa crash	Terminal kasir stabil, state aman	**SUKSES**
6	PembayaranKurangException	Tagihan Rp 3.240.000, konsumen hanya bayar Rp 100.000	Throw PembayaranKurangException, lapor defisit	Status Transaksi TETAP PENDING	**SUKSES**

7	State Recovery Pelunasan	Konsumen serahkan tambahan uang tunai Rp 4.000.000	Pembayaran sukses diterima, kembalian Rp 760.000	Status Transaksi berubah LUNAS	**SUKSES**
8	TransaksiKosongException	Checkout nota TRX-003 tanpa ada item belanja di list	Throw TransaksiKosongException, menolak nota fiktif	Status tetap PENDING, nota kosong	**SUKSES**
9	StatusInvalidException	Kasir tambah item pada TRX-001 yang berstatus LUNAS	Throw StatusTransaksiInvalidException, proteksi data	Status TETAP LUNAS, item tidak berubah	**SUKSES**
10	Rollback Pembatalan Item	Tambah 5 Susu UHT lalu panggil batalkanItem(0)	Item terhapus dari nota, stok 5 unit otomatis dikembalikan	Stok Susu kembali utuh 40 unit	**SUKSES**

8. Jawaban Pertanyaan Evaluasi Modul 10:

a. Perbedaan throw dan throws: Keyword 'throw' digunakan di dalam badan method untuk melempar satu objek instansiasi exception konkret saat anomali terdeteksi (contoh: throw new StokTidakCukupException(...);). Sedangkan 'throws' digunakan pada baris deklarasi signature method untuk mengumumkan kontrak bahwa method tersebut berpotensi meneruskan checked exception yang wajib ditangani dengan try-catch atau dideklarasikan kembali oleh pemanggil (contoh: public void kurangiStok(...) throws StokTidakCukupException).

b. Mengapa pemeriksaan stok dilakukan sebelum pengurangan stok? Untuk menegakkan prinsip Atomisitas dan Konsistensi State (Fail-Safe Invariants). Jika pengurangan dilakukan sebelum pemeriksaan, maka saat kekurangan stok terjadi nilai variabel stok sudah terlanjur berkurang atau negatif di memory runtime. Dengan melakukan validasi di awal (pre-condition check), apabila kondisi tidak terpenuhi eksekusi dibatalkan seketika melalui throw exception tanpa menyentuh baris mutasi stok, menjamin state objek tetap utuh.

c. Akibat jika exception ditangkap tetapi tidak ditangani (swallowed / empty catch): Menyebabkan fenomena 'Silent Failure' yang sangat berbahaya dalam rekayasa perangkat lunak. Program akan mengabaikan terjadinya kegagalan kritis seolah-olah operasi berhasil, sehingga sistem melanjutkan alur eksekusi dengan data yang korup (misal: barang belanjaan tercatat di nota padahal fisiknya tidak ada). Hal ini menghilangkan jejak kesalahan (loss of traceability), menyulitkan debugging, dan dapat mengakibatkan ketidakcocokan inventaris serta audit finansial toko.

d. Mengapa perubahan stok tidak ditempatkan dalam finally? Karena blok 'finally' memiliki sifat mutlak: SELALU dieksekusi baik ketika alur try berhasil maupun saat terjadi kegagalan di blok catch. Jika pengurangan stok ditaruh di finally, maka stok barang akan tetap terpotong meskipun transaksi pembelian sebenarnya ditolak karena uang kurang atau format salah. Blok finally seharusnya hanya digunakan untuk tindakan pembersihan (cleanup) dan verifikasi status objek tanpa memutasi state bisnis.

e. Penjelasan Boundary: Mengapa pembelian tepat sejumlah stok harus diperbolehkan? Secara analisis nilai batas (Boundary Value Analysis), stok bernilai 0 unit adalah state sah (valid domain state) yang menunjukkan barang habis terjual (out-of-stock). Konsumen berhak membeli seluruh unit yang tersedia (jumlah == stok). Batasan bisnis yang dilarang adalah jumlah > stok (yang akan menghasilkan stok negatif). Oleh karena itu, kondisi validasi yang benar adalah 'if (jumlah > stok)' melempar exception, bukan 'jumlah >= stok'.